

Naïve Bayes' Classification

Bayes' theorem

- Bayes' Theorem is a fundamental concept in probability that helps us update the probability of a hypothesis when new evidence becomes available.

- Mathematically

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Meaning of Each Term

- $P(A | B)$ – *Posterior probability*
Probability of event **A** given that **B** has occurred.
- $P(B | A)$ – *Likelihood*
Probability of observing **B** if **A** is true.
- $P(A)$ – *Prior probability*
Initial probability of event **A** before seeing evidence.
- $P(B)$ → Evidence (overall probability of B)

- It represents the conditional probability.
- It means the probability that event A occurs when event B has already occurred.

Example

Intuition behind Baye's theorem

It answers:

"Given that I observed B, how likely is A?"

It **updates prior belief (P(A)) using evidence (B)**.

- Example:

- Suppose:

- 1% of people have a disease
 $P(D) = 0.01$
 - The test detects the disease correctly **99%** of the time
 $P(\text{Pos}|D) = 0.99$
 - $P(\text{Pos}) = 0.02$ (*assume from dataset*)

- $A = D \rightarrow$ person has disease
- $B = \text{Pos} \rightarrow$ test is positive

Given that the test is positive, what is the probability the person actually has the disease?

$$P(D | \text{Pos}) = \frac{P(\text{Pos} | D) P(D)}{P(\text{Pos})}$$

$$\begin{aligned} P(D | \text{Pos}) &= \frac{0.99 \times 0.01}{0.02} \\ &= \frac{0.0099}{0.02} \approx 0.495 \end{aligned}$$

What is Naive Bayes?

- Naive Bayes is a **probabilistic classification algorithm** that applies **Bayes theorem** with the assumption that features are **independent**.
- The probability of a class is computed using the probabilities of the features.
- Advantages:
 - Simple
 - Fast
 - Works well with high-dimensional data
- Note:
 - **Why is it called "Naive"?**
 - The algorithm makes a "**naive**" **assumption** that all features in the dataset are completely **independent** of one another given the class label.

Naïve Bayesian Classification

The naïve Bayesian classifier, or simple Bayesian classifier, works as follows:

1. Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n -dimensional attribute vector, $\mathbf{X} = (x_1, x_2, \dots, x_n)$, depicting n measurements made on the tuple from n attributes, respectively, $A_1, A_2, \dots, A_n$.
2. Suppose that there are m classes, $C_1, C_2, \dots, C_m$. Given a tuple, $\mathbf{X}$, the classifier will predict that $\mathbf{X}$ belongs to the class having the highest posterior probability, conditioned on $\mathbf{X}$. That is, the naïve Bayesian classifier predicts that tuple $\mathbf{X}$ belongs to the class C_i if and only if

$$P(C_i|\mathbf{X}) > P(C_j|\mathbf{X}) \quad \text{for } 1 \leq j \leq m, j \neq i.$$

Thus we maximize $P(C_i|\mathbf{X})$. The class C_i for which $P(C_i|\mathbf{X})$ is maximized is called the *maximum posteriori hypothesis*. By Bayes' theorem (Equation (6.10)),

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}. \quad (6.11)$$

3. As $P(\mathbf{X})$ is constant for all classes, only $P(\mathbf{X}|C_i)P(C_i)$ need be maximized. If the class prior probabilities are not known, then it is commonly assumed that the classes are

equally likely, that is, $P(C_1) = P(C_2) = \dots = P(C_m)$, and we would therefore maximize $P(\mathbf{X}|C_i)$. Otherwise, we maximize $P(\mathbf{X}|C_i)P(C_i)$. Note that the class prior probabilities may be estimated by $P(C_i) = |C_{i,D}|/|D|$, where $|C_{i,D}|$ is the number of training tuples of class C_i in D .

4. Given data sets with many attributes, it would be extremely computationally expensive to compute $P(\mathbf{X}|C_i)$. In order to reduce computation in evaluating $P(\mathbf{X}|C_i)$, the naive assumption of **class conditional independence** is made. This presumes that the values of the attributes are conditionally independent of one another, given the class label of the tuple (i.e., that there are no dependence relationships among the attributes). Thus,

$$\begin{aligned} P(\mathbf{X}|C_i) &= \prod_{k=1}^n P(x_k|C_i) \\ &= P(x_1|C_i) \times P(x_2|C_i) \times \dots \times P(x_n|C_i). \end{aligned} \tag{6.12}$$

We can easily estimate the probabilities $P(x_1|C_i), P(x_2|C_i), \dots, P(x_n|C_i)$ from the training tuples. Recall that here x_k refers to the value of attribute A_k for tuple $\mathbf{X}$. For each attribute, we look at whether the attribute is categorical or continuous-valued. For instance, to compute $P(\mathbf{X}|C_i)$, we consider the following:

- (a) If A_k is categorical, then $P(x_k|C_i)$ is the number of tuples of class C_i in D having the value x_k for A_k , divided by $|C_{i,D}|$, the number of tuples of class C_i in D .
- (b) If A_k is continuous-valued, then we need to do a bit more work, but the calculation is pretty straightforward. A continuous-valued attribute is typically assumed to have a Gaussian distribution with a mean μ and standard deviation σ , defined by

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad (6.13)$$

so that

$$P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i}). \quad (6.14)$$

These equations may appear daunting, but hold on! We need to compute μ_{C_i} and σ_{C_i} , which are the mean (i.e., average) and standard deviation, respectively, of the values of attribute A_k for training tuples of class C_i . We then plug these two quantities into Equation (6.13), together with x_k , in order to estimate $P(x_k|C_i)$. For example, let $X = (35, \$40,000)$, where A_1 and A_2 are the attributes *age* and *income*, respectively. Let the class label attribute be *buys_computer*. The associated class label for X is *yes* (i.e., *buys_computer* = *yes*). Let's suppose that *age* has not been discretized and therefore exists as a continuous-valued attribute. Suppose that from the training set, we find that customers in D who buy a computer are 38 ± 12 years of age. In other words, for attribute *age* and this class, we have $\mu = 38$ years and $\sigma = 12$. We can plug these quantities, along with $x_1 = 35$ for our tuple X into Equation (6.13) in order to estimate $P(\text{age} = 35 | \text{buys_computer} = \text{yes})$. For a quick review of mean and standard deviation calculations, please see Section 2.2.

5. In order to predict the class label of X , $P(X|C_i)P(C_i)$ is evaluated for each class C_i . The classifier predicts that the class label of tuple X is the class C_i if and only if

$$P(X|C_i)P(C_i) > P(X|C_j)P(C_j) \quad \text{for } 1 \leq j \leq m, j \neq i. \quad (6.15)$$

In other words, the predicted class label is the class C_i for which $P(X|C_i)P(C_i)$ is the maximum.

“*How effective are Bayesian classifiers?*” Various empirical studies of this classifier in comparison to decision tree and neural network classifiers have found it to be comparable in some domains. In theory, Bayesian classifiers have the minimum error rate in comparison to all other classifiers. However, in practice this is not always the case, owing to inaccuracies in the assumptions made for its use, such as class conditional independence, and the lack of available probability data.

Bayesian classifiers are also useful in that they provide a theoretical justification for other classifiers that do not explicitly use Bayes’ theorem. For example, under certain assumptions, it can be shown that many neural network and curve-fitting algorithms output the *maximum posteriori* hypothesis, as does the naïve Bayesian classifier.

Class-labeled training tuples from the *AllElectronics* customer database.

<i>RID</i>	<i>age</i>	<i>income</i>	<i>student</i>	<i>credit_rating</i>	<i>Class: buys_computer</i>
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Example 6.4 Predicting a class label using naïve Bayesian classification. We wish to predict the class label of a tuple using naïve Bayesian classification, given the same training data as in Example 6.3 for decision tree induction. The training data are in Table 6.1. The data tuples are described by the attributes *age*, *income*, *student*, and *credit_rating*. The class label attribute, *buys_computer*, has two distinct values (namely, {*yes*, *no*}). Let C_1 correspond to the class *buys_computer* = *yes* and C_2 correspond to *buys_computer* = *no*. The tuple we wish to classify is

$$X = (age = youth, income = medium, student = yes, credit_rating = fair)$$

We need to maximize $P(X|C_i)P(C_i)$, for $i = 1, 2$. $P(C_i)$, the prior probability of each class, can be computed based on the training tuples:

$$P(buys_computer = yes) = 9/14 = 0.643$$

$$P(buys_computer = no) = 5/14 = 0.357$$

To compute $P(X|C_i)$, for $i = 1, 2$, we compute the following conditional probabilities:

$$P(age = youth \mid buys_computer = yes) = 2/9 = 0.222$$

$$P(age = youth \mid buys_computer = no) = 3/5 = 0.600$$

$$P(income = medium \mid buys_computer = yes) = 4/9 = 0.444$$

$$P(income = medium \mid buys_computer = no) = 2/5 = 0.400$$

$$P(student = yes \mid buys_computer = yes) = 6/9 = 0.667$$

$$P(student = yes \mid buys_computer = no) = 1/5 = 0.200$$

$$P(credit_rating = fair \mid buys_computer = yes) = 6/9 = 0.667$$

$$P(credit_rating = fair \mid buys_computer = no) = 2/5 = 0.400$$

Using the above probabilities, we obtain

$$\begin{aligned}P(\mathbf{X} | buys_computer = yes) &= P(age = youth \mid buys_computer = yes) \times \\&\quad P(income = medium \mid buys_computer = yes) \times \\&\quad P(student = yes \mid buys_computer = yes) \times \\&\quad P(credit_rating = fair \mid buys_computer = yes) \\&= 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044.\end{aligned}$$

Similarly,

$$P(\mathbf{X} | buys_computer = no) = 0.600 \times 0.400 \times 0.200 \times 0.400 = 0.019.$$

To find the class, C_i , that maximizes $P(\mathbf{X} | C_i)P(C_i)$, we compute

$$P(\mathbf{X} | buys_computer = yes)P(buys_computer = yes) = 0.044 \times 0.643 = 0.028$$

$$P(\mathbf{X} | buys_computer = no)P(buys_computer = no) = 0.019 \times 0.357 = 0.007$$

Therefore, the naïve Bayesian classifier predicts $buys_computer = yes$ for tuple $\mathbf{X}$.

- Computation of $P(X)$: Why we do not compute $P(x)$

Actual posterior probability= $P(C|X) = \frac{P(X|C)P(C)}{P(X)}$

$P(X)$ is obtained by summing over all classes:

$$P(X) = P(X|yes)P(yes) + P(X|no)P(no)$$

$$P(X|buys_computer = yes)P(buys_computer = yes) = 0.044 \times 0.643 = 0.028$$

$$P(X|buys_computer = no)P(buys_computer = no) = 0.019 \times 0.357 = 0.007$$

$$P(X) = 0.028 + 0.007 = 0.035$$

$$P(yes|X) = \frac{0.028}{0.035} = 0.8$$

$$P(no|X) = \frac{0.007}{0.035} = 0.2$$

In Naïve Bayes classification, we usually do not compute $P(X)$ because it is the same for all classes. Instead we compute:

$$\arg \max_C P(X|C)P(C)$$

which is enough for classification.

1. Using Naïve classifier, predict if Red Domestic SUV will be stolen or not

Example No.	Color	Type	Origin	Stolen?
1	Red	Sports	Domestic	Yes
2	Red	Sports	Domestic	No
3	Red	Sports	Domestic	Yes
4	Yellow	Sports	Domestic	No
5	Yellow	Sports	Imported	Yes
6	Yellow	SUV	Imported	No
7	Yellow	SUV	Imported	Yes
8	Yellow	SUV	Domestic	No
9	Red	SUV	Imported	No
10	Red	Sports	Imported	Yes

2. For the given sample predict whether Tennis shall be played or not.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

{Outlook = sunny, Temperature = cool, Humidity = high, Wind = strong}